

ScAn-Bench: Evaluating Scaling Analysis Methodology

Artin Sermahaj^{1*} Nastaran Alipour^{1,2*} Donat Sinani^{1*} Johannes Hog¹
 Neeratyoy Mallik^{1,2} Steven Adriaensen¹ Jenia Jitsev³ Danny Stoll¹

¹University of Freiburg

²Zuse School ELIZA

³Juelich Supercomputing Center (JSC), Research Center Juelich (FZJ)
 {sermaxha, alipourn, sinanid, stollid}@cs.uni-freiburg.de

Abstract

Recent progress in machine learning is driven by large-scale foundation models, where scaling laws and finding optimal scaling prescriptions for architecture, data, and hyperparameters are key in advancing the state-of-the-art. Therefore, it is surprising that no systematic study evaluates the methodology to obtain scaling laws and prescriptions across different model types. To shed light on this crucial blind spot and facilitate future research, we introduce the surrogate benchmarks ScAn-Bench-LLM and ScAn-Bench-VLM based on 4524 and 8024 checkpoints of language and vision-language model pipelines. On our benchmarks, we perform the first systematic evaluation of both data acquisition and extrapolation methodology for scaling analysis across different data modalities.

1 Introduction

The rapid advancement of foundation models has reshaped approaches to machine learning, driving progress in natural language processing [1–4], computer vision [5, 6], and other fields. As training at target scales is prohibitively expensive, researchers increasingly rely on scaling analysis (ScAn) [7–11] to model empirical relationships between architecture, data, hyperparameters, compute, and loss in order to extrapolate performance, predict optimal training configurations, and derive scaling prescriptions. Determining which configurations to evaluate, how to allocate compute across scales, and how to extrapolate reliably to larger regimes remains a central open problem for ScAn. This challenge is especially acute for less-researched or emerging foundation-model families, where, unlike for large language models (LLMs), established rules of thumb are largely absent.

While recent studies have attempted to compare different ScAn methodologies, these evaluations are overwhelmingly focused on LLMs. More critically, these comparisons often suffer from empirical biases due to inconsistent meta-choices during model training, leading to discrepancies in the reported results even when using the same underlying methodology [12–14]. Existing evaluations also largely study extrapolation on static scaling datasets, while evaluating data acquisition strategies requires interactive and reproducible benchmarks.

In other meta-algorithmic fields such as neural architecture search (NAS) and hyperparameter optimization (HPO), which face similar challenges [15], tabular and surrogate benchmarks have emerged as key enabling research artefacts for efficient evaluation. By collecting a representative dataset of trained configurations, one can either precompute outcomes for a small search space exhaustively or fit a fast surrogate predictor over this data to approximate the outcome of expensive

*Equal contribution. Ordering among equally contributing authors determined by roulette.

training runs in rich search spaces in seconds. A prominent example of the former is NAS-Bench-101 [16], while surrogate benchmarks have since emerged as a widely adopted approach for large or continuous domains [17–23].

Thus, to enable the systematic study of ScAn methodology, we introduce the two surrogate benchmarks ScAn-Bench-VLM and ScAn-Bench-LLM, covering vision-language models (VLMs) and LLMs respectively. We evaluate 9 scaling and hyperparameter settings over 1,000 training runs for each benchmark, resulting in 8,024 and 4,524 checkpoints for VLMs and LLMs, respectively, with model sizes ranging from 2M to 357M parameters for VLMs and from 16M to 1B for LLMs. We fit this data to our surrogate predictor, modeling the training dynamics and downstream performance, enabling accurate approximation of training runs in CPU-seconds rather than GPU-hours.

We demonstrate that our benchmarks make it possible to study ScAn methods under reproducible conditions, presenting the first apples-to-apples comparison between existing methods across multiple data modalities and foundation model families. Beyond our evaluation, our benchmarks will allow even researchers without access to large-scale GPU clusters to advance ScAn methodology.

Our main contributions are:

- **Scaling analysis framework:** We develop a systematic view on scaling analysis to guide benchmarking and evaluation (Section 2).
- **Surrogate benchmarks:** We introduce the first surrogate benchmarks (ScAn-Bench-LLM and ScAn-Bench-VLM) suitable for research on scaling analysis methodology (Section 3).
- **Evaluation of scaling analysis methodology:** We provide the first controlled evaluation of both sequential data acquisition and extrapolation strategies across multiple foundation model families (Section 4).

For our code see: https://github.com/automl/scan_bench.

2 A systematic view on scaling analysis

Existing scaling analysis methodologies differ not only in their extrapolation models, but also in how training configurations are sampled, compute is allocated, and optimization hyperparameters are chosen, and even what quantities are extrapolated [8, 24, 25]. Recent studies have shown that these implementation choices can substantially affect empirical scaling conclusions [12, 13]. To enable systematic evaluation, we introduce a unified framework that decomposes scaling analysis into a *data acquisition* phase under a constrained search budget and an *extrapolation* phase that predicts optimal behavior at unseen compute scales.

The scaling problem We consider a pretraining task loss, f , to be minimized at a target compute budget C_{target} (typically FLOPs). Given a joint hyperparameter search space $\Lambda = \Lambda_c \times \Lambda_o$, where Λ_c is the space of compute-scaling hyperparameters and Λ_o is the space of non-scaling optimization hyperparameters, our theoretical goal is to find the optimal configuration $(\lambda_c^*, \lambda_o^*)$ that minimizes the objective function subject to this final compute constraint. Note that the compute budget is strictly a function of the compute-scaling hyperparameters, $C(\lambda_c)$. This problem can be formally stated as:

$$(\lambda_c^*, \lambda_o^*) = \arg \min_{(\lambda_c, \lambda_o) \in \Lambda} f(\lambda_c, \lambda_o) \quad \text{subject to} \quad C(\lambda_c) \leq C_{\text{target}} \quad (1)$$

Scaling analysis methodology approximates this theoretical solution through a constrained data acquisition phase followed by an extrapolation phase.

Data acquisition A data acquisition strategy samples a set of configurations and evaluates them to yield a dataset $\mathcal{D} = \{(\lambda_{c,i}, \lambda_{o,i}, f(\lambda_{c,i}, \lambda_{o,i}))\}_{i=1}^M$. The sampling process is bounded by a total allotted search budget $\sum_{i=1}^M C(\lambda_{c,i}) \leq C_{\text{search}}$, which in practice is usually not enough to run adequate evaluations on the target compute C_{target} . A fundamental challenge in standardizing the evaluations is decoupling the data acquisition strategy from domain-specific heuristics. For instance, the original Chinchilla [12] methodologies implicitly rely on heavily tuned prior knowledge for non-scaling hyperparameters, λ_o (e.g., learning rate schedules). To ensure a rigorous, prior-free comparison of the allocation strategies, we must standardize how these baselines are instantiated.

Table 1: Runtime comparison for 100 evaluations. Surrogate querying is measured in CPU hours on AMD EPYC 9655 CPU using 8 cores, while training cost is reported in GPU hours on A100 GPUs.

Model family	Surrogate query (CPU-s)	Training (GPU-h)
VLM	29	419.58
LLM	17	3519.42

Loss extrapolation Utilizing the empirically acquired dataset $\mathcal{D}$, the most fundamental layer of extrapolation applies a predictive technique for the optimal loss. Rather than attempting to model the entire multi-dimensional loss landscape, the goal is to extrapolate the objective function f along the empirically observed Pareto optimal frontier, directly predicting the minimal achievable loss for a given target compute budget C_{target} .

Coarse parameter extrapolation Beyond predicting raw loss, many existing scaling studies reduce problem complexity by treating the overall model size N as a single scalar proxy for scale. This intermediate layer assumes that once the optimal N is determined, the underlying hyperparameters can be derived via established domain heuristics [12, 24]. Under this regime, the extrapolation specifically models the model size N^* and consequently optimal token count D^* .

Full extrapolation The most fine-grained layer of extrapolation eliminates the reliance on heuristic priors by providing a direct predictive prescription for the complete configuration. In this underexplored setting, the fully-parameterized extrapolation function, $\mathcal{Q}_\Lambda$, directly models the optimal joint set of compute-scaling and non-scaling optimization hyperparameters $(\hat{\lambda}_c^*, \hat{\lambda}_o^*)$ as a function of the target compute budget C_{target} .

3 ScAn-Bench-LLM and ScAn-Bench-VLM

Training and evaluating foundation models is computationally expensive, making direct evaluation of ScAn methods unfeasible for many research labs, hindering progress in this area. To address this, we construct surrogate benchmarks that approximate the mapping between training configurations and performance. While surrogate predictions are obtained quickly on CPU, training based evaluation requires hours on GPUs and is orders of magnitude slower (see Table 1). Unlike static tabular benchmarks, surrogate benchmarks can be queried outside the collected configurations, allowing for realistic design spaces and continuous parameters.

Section 3.1 describes the ScAn spaces and data collection pipelines for VLM and LLM settings, while Section 3.2 presents the surrogate modeling procedure. We provide additional details in Appendix A and Appendix B.

3.1 ScAn spaces and data collection

We define two training pipelines with corresponding ScAn spaces for the considered model families. The VLM pipeline trains CLIP [5] models using an open-source implementation [9] on a 60M-sample subset of LAION-400M [26]. The LLM pipeline trains decoder-only transformers [27] on the SlimPajama [28] dataset.

ScAn space For each model family, we define a bounded ScAn space spanning both scaling variables and optimization hyperparameters. Table 2 summarizes the number of continuous and discrete parameters considered across model families, while Appendix A.1 provides the full parameter ranges and domains. For VLMs, scaling variables include the vision and text encoder width together with the number of training samples. For LLMs, we vary the number of layers, number of attention heads, embedding dimension, and training tokens. Across all model families, we additionally vary key optimization hyperparameters. For VLMs, these include the learning rate, weight decay, warmup fraction, and optimizer coefficients $(\beta_1, \beta_2, \epsilon)$. For LLMs, we vary the learning rate, weight decay, cooldown steps, and optimizer coefficients (β_1, β_2) .

Table 2: Summary of the two surrogate benchmarks. The table reports the number of hyperparameters (#HPs), scale parameters (#Scale), FLOP ranges, continuous and discrete dimensions, and the number of downstream tasks (#Down.) modeled by each surrogate benchmark.

Benchmark	#HPs	#Scale	FLOP Range	Cont.	Disc.	#Down.
ScAn-Bench-VLM	6	3	$3.6 \times 10^{13} - 7.8 \times 10^{18}$	7	2	40
ScAn-Bench-LLM	5	4	$3.2 \times 10^{16} - 2.9 \times 10^{20}$	6	3	-

Table 3: Comparison of surrogate performance across model families in predicting upstream performance, measured by Spearman rank correlation (higher is better) and RMSE (lower is better).

Surrogate	VLM		LLM	
	Spearman $\uparrow$	RMSE $\downarrow$	Spearman $\uparrow$	RMSE $\downarrow$
TabPFN	0.98	0.21	0.95	0.27
AutoGluon	0.98	0.26	0.91	0.30
Ensemble (XGB)	0.96	0.35	0.85	0.35
Ensemble (Mix)	0.95	0.37	0.86	0.33
Ensemble (LGB)	0.96	0.42	0.87	0.33

Data collection To capture training dynamics and allow queries at intermediate training stages, we recorded a total of 8024 checkpoints for VLMs and 4524 checkpoints for LLMs. To limit storage overhead, we employed a controlled checkpointing scheme that avoids excessive checkpoint density while preserving information of the training trajectory. Details on the checkpointing strategy are provided in Appendix A.3. For each run, we record upstream and downstream performance, evaluating on all checkpoints. We provide a complete overview of the recorded metrics and the model pipelines in Appendix A.4.

3.2 Creating the surrogate benchmarks

We follow existing literature on surrogate benchmark construction [23] for surrogate model candidates and evaluation procedures.

Surrogate modeling We consider AutoGluon [29], XGBoost [30], LightGBM [31], TabPFN, and a mixed ensemble consisting of Linear Regression, Ridge Regression [32], Random Forests [33], XGBoost and LightGBM models. In addition, we also include TabPFN [34] as a recent modeling approach for tabular setting, which we find to perform best. More information on each surrogate is provided in the Appendix B.1.

Model selection Model selection is based on each surrogate’s ability to predict upstream performance. Table 3 reports surrogate performance on the held out test set in terms of Spearman rank correlation and RMSE across all model families. TabPFN consistently yields the strongest performance across benchmarks and is therefore selected as the default surrogate predictor. Additional information on data split is provided in Appendix B.2. A more comprehensive evaluation across additional metrics is provided in Appendix B.3.

Final model and runtime We use TabPFN as the predictor for both model families, trained on all available data, and model each target metric independently, including upstream and downstream metrics. Table 2 summarizes the properties of the surrogate benchmarks. In addition to the performance surrogate, we train a separate binary surrogate for VLMs that predicts whether a configuration will diverge. Further details are provided in Appendix B.4. The predictors are integrated into a broader API that exposes additional configuration-level information (Table 13), with further details in Appendix C.

4 A systematic evaluation of ScAn methodology

As introduced in Section 2, scaling analysis requires two distinct methodological choices: a data acquisition strategy to gather data at lower compute scales, and an extrapolation strategy to project those observations to a target compute regime. To systematically benchmark these components, we evaluate several standard approaches from the literature for both phases. Using the ScAn-Bench surrogate benchmarks introduced in Section 3, we conduct a scaling analysis to answer the following questions:

- RQ1:* What are the trade-offs during data acquisition among maximizing immediate performance, broadly exploring the Pareto frontier, and ensuring a highly accurate scaling law fit?
- RQ2:* Is there a combination of loss extrapolation technique and data acquisition that consistently performs well across benchmarks?
- RQ3:* Is there a combination of coarse parameter extrapolation technique and data acquisition that consistently performs well across benchmarks?
- RQ4:* Does the studied full extrapolation approach provide reliable and accurate results?
- RQ5:* Do different acquisition approaches systematically favor particular extrapolation techniques?

4.1 Evaluated ScAn Methodology

Data acquisition strategies As established in Section 2, decoupling strategies from domain-specific heuristic is essential to have a valid comparison between strategies across different domains. To this end, we adapt 4 generalized heuristic-free approaches. From the Chinchilla [8] study, we evaluate Iso-Parameter Profiling (Chinchilla Approach 1) and Iso-FLOP Profiling (Chinchilla Approach 2), both of which rely on deterministic, grid-based sampling strategies. To evaluate dynamic sampling, we include Cost-Aware Robust Bayesian Search (CARBS) [25], which utilizes an adapted EI acquisition function to sequentially sample configurations based on previous observations. Finally, we include Random Search as a baseline to isolate the performance gains of the structured allocation strategies. The design choices for these approaches can be found in Appendix E.

Loss extrapolation The coarsest level, which directly predicts the minimal loss at the target compute scale using either a Kaplan-style power law fit [12] or a Chinchilla-style parametric loss function [8].

Coarse parameter extrapolation This level predicts the optimal compute-scaling parameters, specifically the model size (N^*) and the token count (D^*). Here, we evaluate deriving N^* and D^* via the Chinchilla parametric loss estimate under cost constraints, as well as via direct power law fitting between the macro-parameters and total compute.

Full extrapolation The most fine-grained level models both λ_c^* and λ_o^* at the target scale. As this setting remains largely unexplored, we include a representative baseline based on independent linear regression (as in CARBS), leaving more expressive joint approaches for future work.

4.2 Experimental setup

Hyperparameter space Our data acquisition phase is strictly confined to a lower-compute regime bounded by $C_{\max}$. To rigorously evaluate the performance of the extrapolation techniques, we define the unobserved target compute budget as $C_{\text{target}} = 20 \times C_{\max}$. By anchoring C_{target} within the compute scale of our benchmark datasets, we ensure the availability of ground-truth evaluations at the target scale. This prevents reliance on an unverified extrapolative regime, allowing us to precisely measure the true extrapolation error. While our overall hyperparameter search space Λ is defined in compliance with these existing benchmarks, the specific configurations sampled during data acquisition are strictly restricted to configurations costing less than $C_{\max}$. Specifically, we establish $C_{\max} = 2 \times 10^{16}$ FLOPs for the OpenCLIP benchmark and $C_{\max} = 1.0 \times 10^{19}$ FLOPs for the LLM benchmark.

Loss function In line with scaling law literature [8, 24], we establish pretraining validation loss as our objective function, f . While downstream task performance is frequently used for general model

evaluation, standard extrapolation methodologies are predominantly formulated around validation loss. This is because validation loss provides a monotonically decreasing signal that is well-suited to power-law fitting and parametric modeling. By strictly isolating validation loss, we ensure our methodological evaluation is decoupled from the unpredictable dynamics of downstream tasks.

Seed Aggregation To mitigate the impact of variance from our random hyperparameter initialization, we conducted the experiment in 10 independent seeds. Throughout Section 4.3, we report the mean performance across these runs, along with the standard error to quantify predictive stability.

4.3 Results

Data acquisition As established at the start of Section 4 the primary objective of this evaluation is to systematically analyze the trade-offs between different data acquisition strategies and prediction methodologies. In doing so, we demonstrate how notoriously sensitive scaling law projections are to underlying design choices. Figure 1 evaluates performance across three metrics: Pareto Estimation Regret (PER, top) measures the area between the acquired data’s fit and the true Pareto frontier, Hypervolume (middle) measures broad spatial coverage and Incumbent Loss (bottom) tracks the best loss achieved over the search trajectory. Observing the results, a clear exploration-exploitation trade-off emerges. CARBS efficiently exploits high-performing regions, achieving the lowest PER and superior early-stage Incumbent Loss. However, the deterministic Iso-Parameter approach ensures broad exploration, yielding the highest Hypervolume. Tracking the best loss found as the compute budget expands, RS surpasses CARBS across benchmarks. This suggests that while CARBS excels early, its exploitation bias restricts its search space during later acquisition phases. Figure 2 plots the scaling law fits and the empirical Pareto frontier for a single seed, directly illustrating the localized nature of the data acquired via CARBS.

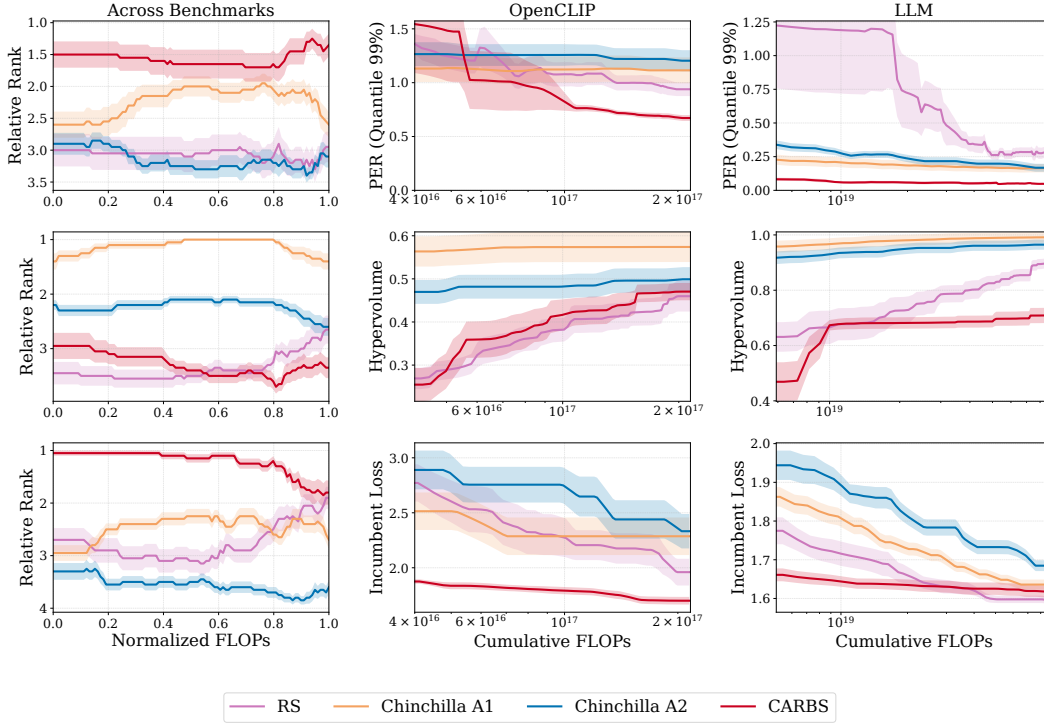


Figure 1: **Standalone evaluation of data acquisition strategies.** The performance of 4 different data acquisition strategies within the search space across three metrics against the cumulative search budget: PER (top, measuring proximity to the true optimal frontier), Hypervolume (middle, measuring broad spatial exploration), and Incumbent Loss (bottom, tracking the best loss found).

Loss extrapolation We assess loss-level extrapolation techniques under both Iso-Parameter Profiling (Chinchilla A1) and CARBS data acquisition by examining their predicted loss at C_{target}

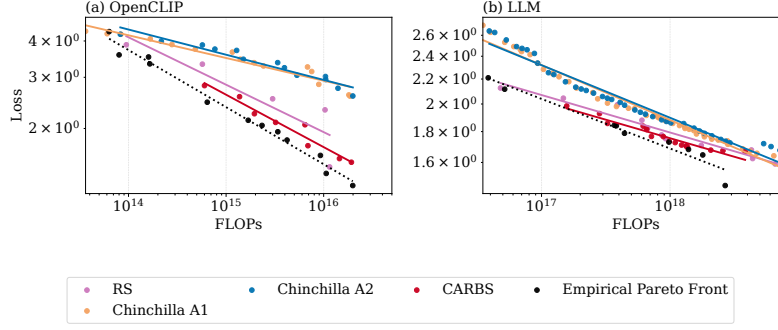


Figure 2: **Power law fit across acquisition strategies.** Projected power law fits for (a) OpenCLIP and (b) LLM for a single selected random seed, evaluated at the maximum search budget and plotted alongside the true empirical Pareto optimal frontier. Notably, while the data points acquired via CARBS exhibit a highly localized and narrow distribution, the resulting scaling trajectory closely mirrors the optimal empirical trend.

as a function of cumulative compute. Our findings indicate that no single data acquisition and extrapolation combination is optimal across all benchmarks.

Under Iso-Parameter Profiling (Figure 3, Chinchilla A1), the parametric loss model (Chinchilla Approach 3) achieves closer predictions to the empirical target loss, while Kaplan exhibits more stable, monotonic improvement with increasing compute. Figure 4 illustrates this same comparison when data is acquired via the CARBS strategy. Chinchilla Approach 3 significantly outperforms Kaplan in anytime prediction on the OpenCLIP benchmark. Conversely, Kaplan yields more reliable and accurate predictions on the LLM benchmark under this acquisition regime.

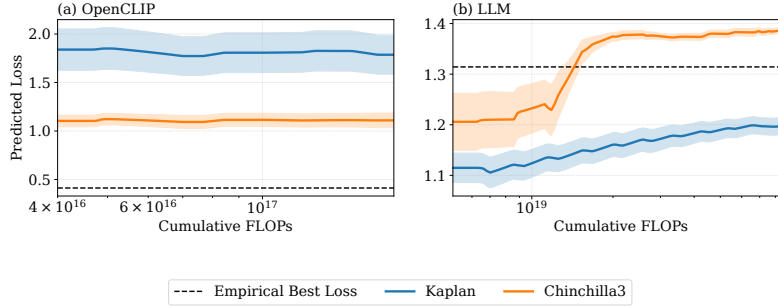


Figure 3: **Comparison of loss extrapolation techniques under Iso-Parameter Profiling (Chinchilla A1).** Predicted loss trajectories over cumulative FLOPs for the (a) OpenCLIP and (b) LLM benchmarks. The plots compare the Chinchilla Approach 3 and Kaplan prediction methods against the empirical true optimal loss (dashed line) at C_{target} . While the Chinchilla technique generally achieves better overall performance across the trajectory, the Kaplan method demonstrates a more stable, monotonic improvement in its predictions as the compute budget scales over time.

Coarse parameter extrapolation Parametric extrapolation models are fundamentally brittle when their underlying compute assumptions are violated. A primary limitation of using the Chinchilla Approach 3 to model optimal model size (N^*) is its strict reliance on the standard $C \approx 6ND$ compute estimation. To strictly evaluate the original formulation without injecting novel correction heuristics for multi-modal architectures, we held this standard compute estimation constant across all benchmarks. As demonstrated in Figure 5(a) under Iso-Parameter profiling, this rigid assumption causes the Chinchilla technique to fail in predicting the optimal OpenCLIP model size. Conversely, Figure 5(b) confirms that both the Chinchilla and empirical Power-Law projection methods reliably predict N^* for the standard LLM benchmark. This contrast highlights that the purely empirical Power-Law Fit is a more robust and generalized estimator across diverse architectural domains. We therefore isolate the performance of the Power-Law approach across data acquisition strategies in Figure 6. Except for CARBS, which yields poor predictions due to its localized exploitation of the

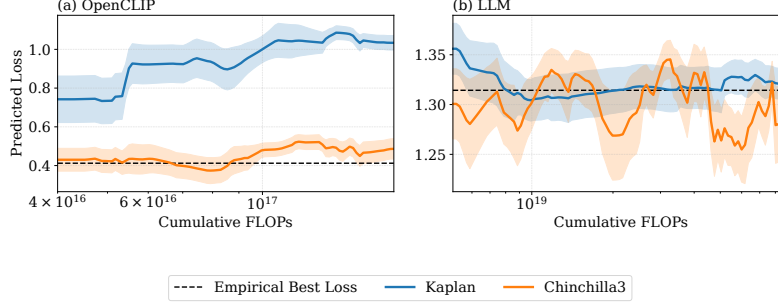


Figure 4: **Comparison of loss extrapolation techniques under CARBS data acquisition.** Predicted loss trajectories over cumulative FLOPs against the empirical true optimal loss (dashed line) for the (a) OpenCLIP and (b) LLM benchmarks.

search space, other acquisition methodologies have accurate predictions of N^* at the given search budget.

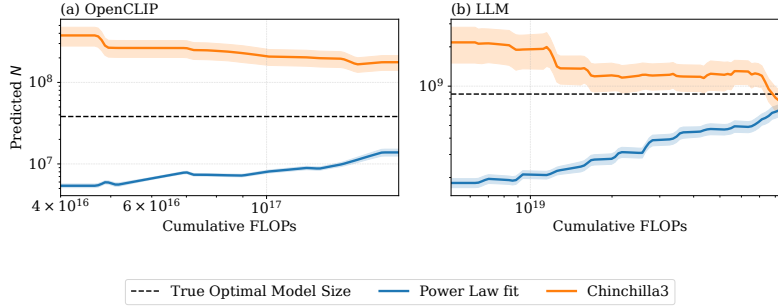


Figure 5: **Model size extrapolation under Iso-Parameter data acquisition.** Trajectories of the predicted optimal model size (N^*) for the (a) OpenCLIP and (b) LLM benchmarks. The parametric Chinchilla approach fails on OpenCLIP due to invalid compute estimation heuristics, whereas the purely empirical Power-Law fit remains robust across both domains.

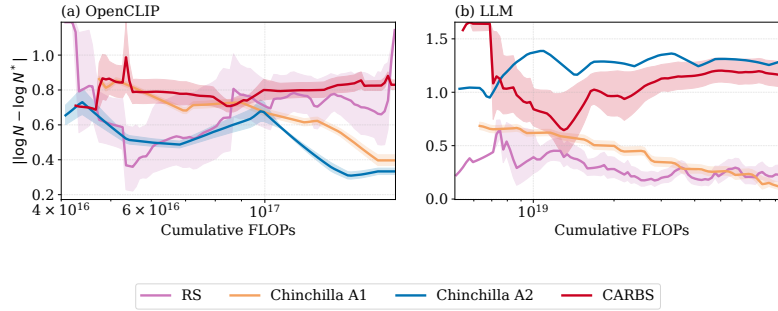


Figure 6: **Robustness of Power-Law model size predictions across acquisition strategies.** The y-axis represents the absolute prediction error, calculated as the absolute difference between the log-scale of the predicted model size and the true empirical optimal model size ($|\log_{10}(N_{\text{pred}}) - \log_{10}(N_{\text{true}}^*)|$) at the target compute budget C_{target} . With the exception of CARBS, most of the profiling strategies facilitate accurate predictions of N^* at the given search budget.

Full parameter extrapolation Predicting the full set of hyperparameters for a given compute budget, C_{target} , represents the most direct solution to the scaling law problem. As detailed in Section 4, the specific full parameter extrapolation strategy we evaluate involves fitting independent linear regression models to each hyperparameter dimension. This regression fits the empirically observed

Table 4: **Actual loss at predicted configurations.** Evaluation of full-parameter extrapolations using CARBS and Random Search (RS) at various C_{target} budget ratios.

Strategy	LLM			OpenCLIP		
	0.1x	0.5x	Best	0.1x	0.5x	Best
CARBS	2.02 ± 0.23	1.76 ± 0.17	1.31	2.92 ± 0.41	2.49 ± 0.14	0.41
RS	1.71 ± 0.18	1.89 ± 0.37	1.31	2.28 ± 0.29	1.81 ± 0.30	0.41

Pareto optimal points as inputs, a methodology used by CARBS [25]. Table 4 compares the actual loss achieved by these predicted configurations ($\hat{\lambda}_c^*$, $\hat{\lambda}_o^*$) against the best empirical loss at C_{target} .

For this extrapolation experiment, we isolate our analysis to CARBS and Random Search acquisition strategies as they dynamically sample the non-scaling hyperparameters (λ_o) during the acquisition phase, whereas the deterministic grid-based methods hold λ_o fixed across a given experiment. We evaluate the prediction quality across varying search budgets, specifically $C_{\text{search}} \in \{0.1, 0.5\} \times C_{\text{target}}$. As shown in Table 4, as the search budget increases, the standard error across different random seeds decreases, indicating more stabilized extrapolation over time. Notably, Random Search outperforms CARBS across all evaluated budgets. Furthermore, the accuracy of the Random Search predictions monotonically improves by yielding lower actual losses as more data is acquired. Crucially, this approach is highly sensitive to the exact composition of the empirical Pareto frontier. This naive method assumes dimensional independence, ignoring the coupled scaling correlations between hyperparameters. For example, the predicted OpenCLIP configuration failed to account for interactions between the text and image tower hyperparameters, resulting in severely degraded performance.

Interaction between data acquisition and prediction For loss extrapolation, the parametric model (Chinchilla 3) consistently outperforms empirical Power-Law fits (Figure 14). This advantage stems from its highly flexible five-parameter formulation, which effectively captures the irreducible loss and accurately fits the acquired data. However, this flexibility introduces severe identifiability issues: the optimization can accurately represent the loss surface without identifying the true underlying data-generating parameters. Consequently, this success does not translate to optimal model size (N^*) prediction. Because the parametric derivation of N^* strictly depends on the precise ratio of specific fitted coefficients, it is highly brittle under poor identifiability. Instead, the efficacy of N^* prediction is dictated by whether the data acquisition phase explicitly traces a usable compute-optimal frontier. Structured profiling methods (e.g., Chinchilla A1/A2) are designed to map this envelope, making the empirical Power-Law approach highly accurate and efficient. Conversely, unstructured methods like Random Search broadly scatter (N, D) allocations without targeting the envelope, while highly exploitative methods like CARBS sample too narrowly. These degenerate traces fail to map the Pareto frontier, making the Power-Law approach ineffective and leaving the parametric fit—despite its identifiability caveats—as the only remaining viable option (Figure 15).

5 Related benchmarks and evaluations

Benchmarks A large body of work studies benchmarking for architecture search [16, 17, 19, 21, 35–41], hyperparameter optimization [18, 42–45], and joint hyperparameter and architecture search [20, 22, 46, 47]. These benchmarks typically focus on smaller MLP or CNN architectures, often assume fixed training budgets, or do not provide suitable scaling parameters. Recent work has explored transformer-based models, but does not model independent training runs or hyperparameter variation [23]. To support benchmarking ScAn methods, we provide rich scale and hyperparameter spaces for VLM and LLM settings, support querying upstream metrics, downstream task performance, and report other necessary statistics such as FLOPs and parameter count.

Evaluations in the language domain For LLMs, recent efforts have begun to evaluate the methods used to derive scaling laws. For instance, Porian et al. [13] reveal that discrepancies in optimal scale predictions often stem from inconsistent parameter and compute counting heuristics. Similarly, Choshen et al. [48] highlight the high variance and methodological pitfalls of extrapolating scaling laws from intermediate training checkpoints rather than fully converged models. Muennighoff et al.

[49] isolate the data axis, demonstrating that training under data-constrained regimes with repeated corpora yields similar power-law dynamics, albeit with diminishing returns. Moving beyond scalar model size, McLeish et al. [50] investigate the impact of architectural shape over the compute horizon, however, their evaluation remains restricted to a highly constrained pool of model sizes.

6 Conclusion

To facilitate the systematic study of ScAn methodology and make it computationally feasible, we introduced the two surrogate benchmarks, ScAn-Bench-VLM and ScAn-Bench-LLM, and evaluated existing scaling methodologies. We hope ScAn-Bench enables the community to develop and evaluate new ScAn methodology in a more systematic and reproducible manner, and that the benchmarks we provide serve as a template and encourage others to create additional benchmarks.

Limitations Our benchmarks provide an initial framework for studying the scaling problem, the compute scales, however, remain substantially smaller than current state-of-the-art models. Consequently, the observed trends may not fully transfer to significantly larger-scale regimes. We do not evaluate the sensitivity of the ScAn methods to their hyperparameters. VLM and LLM pipelines have substantially different characteristics, so, where results align across these settings, this provides some evidence that the finding may extend beyond a single pipeline. However, as our evaluation also shows, results do not always align, and they may not even align across different architectures and training recipes for the same data modality. Since evaluations of ScAn methodology typically consider only a single setting, this highlights the risk of drawing overly general conclusions from one pipeline. Similar to how benchmarking progressed in other meta-algorithmic fields, we hope our work will motivate future benchmarks across additional model families, architectures, and training recipes. Finally, we restrict our analysis to upstream scaling behavior, but note that extending the evaluation to downstream tasks is supported by our surrogate benchmarks.

Acknowledgments

This research was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under grant number 539134284, through EFRE (FEIH_2698644) and the state of Baden-Württemberg. The authors gratefully acknowledge the computing time made available to them on the high-performance computers and at the NHR Centers at TU Dresden and KIT. These centers are jointly supported by the Federal Ministry of Research, Technology and Space of Germany and the state governments participating in the NHR (www.nhr-verein.de/unsere-partner). The authors gratefully acknowledge the Gauss Center for Supercomputing eV (www.gauss-centre.eu) for funding this project by providing computing time through the John von Neumann Institute for Computing (NIC) on the GCS Supercomputer JUWELS at Jülich Supercomputing Center (JSC). The authors acknowledge funding by the European Union (via ERC Consolidator Grant DeepLearning 2.0, grant no. 101045765). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them. Neeratyoy Mallik and Nastaran Alipour are supported by the Konrad Zuse School of Excellence in Learning and Intelligent Systems (ELIZA) through the DAAD programme “Konrad Zuse Schools of Excellence in Artificial Intelligence”, sponsored by the Federal Ministry of Education and Research. Johannes Hog acknowledges funding by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under SFB 1597 (SmallData), grant number 499552394.



Baden-Württemberg



Funded by
the European Union

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Advances in Neural Information Processing Systems (NeurIPS'17)*. Curran Associates, Inc., 2017.
- [2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.-F. Balcan, and H. Lin, editors, *Proceedings of the 33rd International Conference on Advances in Neural Information Processing Systems (NeurIPS'20)*, pages 1877–1901. Curran Associates, 2020.
- [3] OpenAI. Gpt-4 technical report. *arXiv:2303.08774 [cs.CL]*, 2023.
- [4] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081): 633–638, 2025.
- [5] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. *arXiv:2103.00020v1 [cs.CV]*, 2021.
- [6] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. HAZIZA, F. Massa, A. El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.
- [7] J. Kaplan, S. McCandlish, T. Henighan, T. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv:2001.08361 [cs.LG]*, 2020.
- [8] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. Training compute-optimal large language models. In *Proceedings of the 35th International Conference on Advances in Neural Information Processing Systems (NeurIPS'22)*, 2022.
- [9] M. Cherti, R. Beaumont, R. Wightman, M. Wortsman, G. Ilharco, C. Gordon, C. Schuhmann, L. Schmidt, and J. Jitsev. Reproducible scaling laws for contrastive language-image learning. In *Proceedings of the International Conference on Computer Vision and Pattern Recognition (CVPR'23)*. Computer Vision Foundation and IEEE Computer Society, IEEE, 2023.
- [10] M. Nezhurina, T. Porian, G. Puccetti, T. Keressies, R. Beaumont, M. Cherti, and J. Jitsev. Scaling laws for robust comparison of open foundation language-vision models and datasets. In *Advances in Neural Information Processing Systems*, 2025.
- [11] H. Li, W. Zheng, Q. Wang, H. Zhang, Z. Wang, S. Xuyang, Y. Fan, Z. Ding, H. Wang, N. Ding, S. Zhou, X. Zhang, and D. Jiang. Predictable scale: Part i – optimal hyperparameter scaling law in large language model pretraining. *arXiv preprint arXiv:2503.04715*, 2025.
- [12] T. Pearce and J. Song. Reconciling kaplan and chinchilla scaling laws, 2024.
- [13] T. Porian, M. Wortsman, J. Jitsev, L. Schmidt, and Y. Carmon. Resolving discrepancies in compute-optimal scaling of language models, 2025.
- [14] M. Li, S. Kudugunta, and L. Zettlemoyer. (mis)fitting: A survey of scaling laws, 2025.
- [15] A. Yang, P. Esperança, and F. Carlucci. NAS evaluation is frustratingly hard. In *The Eighth International Conference on Learning Representations (ICLR'20)*, 2020.

- [16] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, and F. Hutter. NAS-Bench-101: Towards reproducible Neural Architecture Search. In K. Chaudhuri and R. Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning (ICML'19)*, volume 97, pages 7105–7114. Proceedings of Machine Learning Research, 2019.
- [17] J. Siems, L. Zimmer, A. Zela, J. Lukasik, M. Keuper, and F. Hutter. NAS-bench-301 and the case for surrogate benchmarks for Neural Architecture Search. *arXiv:2008.09777v4 [cs.LG]*, 2020.
- [18] K. Eggensperger, P. Müller, N. Mallik, M. Feurer, R. Sass, A. Klein, N. Awad, M. Lindauer, and F. Hutter. HPOBench: A collection of reproducible multi-fidelity benchmark problems for HPO. In J. Vanschoren and S. Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. Curran Associates, 2021.
- [19] S. Yan, C. White, Y. Savani, and F. Hutter. NAS-bench-x11 and the power of learning curves. In M. Ranzato, A. Beygelzimer, K. Nguyen, P. Liang, J. Vaughan, and Y. Dauphin, editors, *Proceedings of the 34th International Conference on Advances in Neural Information Processing Systems (NeurIPS'21)*, volume 34, pages 22534–22549. Curran Associates, 2021.
- [20] Y. Hirose, N. Yoshinari, and S. Shirakawa. NAS-HPO-Bench-II: A benchmark dataset on joint optimization of convolutional neural network architecture and training hyperparameters. In V. Balasubramanian and I. Tsang, editors, *Proceedings of the 13th Asian Conference on Machine Learning (ACML)'21*, volume 157, pages 1349–1364. PMLR, 2021.
- [21] A. Zela, J. Siems, L. Zimmer, J. Lukasik, M. Keuper, and F. Hutter. Surrogate NAS benchmarks: Going beyond the limited search spaces of tabular NAS benchmarks. In *The Tenth International Conference on Learning Representations (ICLR'22)*. ICLR, 2022.
- [22] A. Bansal, D. Stoll, M. Janowski, A. Zela, and F. Hutter. JAHS-bench-201: A foundation for research on joint architecture and hyperparameter search. In *Proceedings of the 35th International Conference on Advances in Neural Information Processing Systems (NeurIPS'22)*, 2022.
- [23] R. S. Sukthanker, A. Zela, B. Staffler, A. Klein, L. Purucker, J. K. H. Franke, and F. Hutter. Hw-gpt-bench: Hardware-aware architecture benchmark for language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Proceedings of the 37th International Conference on Advances in Neural Information Processing Systems (NeurIPS'24)*. Curran Associates, 2024.
- [24] J. Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [25] A. J. Fetterman, E. Kitanidis, J. Albrecht, Z. Polizzi, B. Fogelman, M. Knutins, B. Wróblewski, J. B. Simon, and K. Qiu. Tune as you scale: Hyperparameter optimization for compute efficient training, 2023.
- [26] C. Schuhmann, R. Vencu, R. Beaumont, R. Kaczmarczyk, C. Mullis, A. Katta, T. Coombes, J. Jitsev, and A. Komatsuzaki. Laion-400m: Open dataset of clip-filtered 400 million image-text pairs, 2021.
- [27] N. Ajroldi. plainlm: Language model pretraining in pytorch. <https://github.com/Niccolo-Ajroldi/plainLM>, 2024.
- [28] D. Soboleva, F. Al-Khateeb, R. Myers, Jacob R S., J. Hestness, and N. Dey. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama. <https://cerebras.ai/blog/slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama>, 2023.
- [29] N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, and A. Smola. Autogluon-tabular: Robust and accurate automl for structured data. *arXiv:2003.06505 [stat.ML]*, 2020.

- [30] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In B. Krishnapuram, M. Shah, A. Smola, C. Aggarwal, D. Shen, and R. Rastogi, editors, *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'16)*, pages 785–794. ACM Press, 2016.
- [31] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Proceedings of the 31st International Conference on Advances in Neural Information Processing Systems (NeurIPS'17)*, 2017.
- [32] A. E. Hoerl and R. W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- [33] L. Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [34] L. Grinsztajn, K. Flöge, O. Key, F. Birkel, P. Jund, B. Roof, B. Jäger, D. Safaric, S. Alessi, A. Hayler, M. Manium, R. Yu, F. Jablonski, S. Bin Hoo, A. Garg, J. Robertson, M. Bühler, V. Moroshan, L. Purucker, C. Cornu, L. Charlotte Wehrhahn, A. Bonetto, B. Schölkopf, S. Gambhir, N. Hollmann, and F. Hutter. Tabpfn-2.5: Advancing the state of the art in tabular foundation models, 2026.
- [35] X. Dong and Y. Yang. NAS-Bench-201: Extending the scope of reproducible Neural Architecture Search. In *The Eighth International Conference on Learning Representations (ICLR'20)*, 2020.
- [36] A. Zela, J. Siems, and F. Hutter. NAS-Bench-1Shot1: Benchmarking and dissecting One-shot Neural Architecture Search. In *The Eighth International Conference on Learning Representations (ICLR'20)*, 2020.
- [37] N. Klyuchnikov, I. Trofimov, E. Artemova, M. Salnikov, M. Fedorov, A. Filippov, and E. Burnaev. Nas-bench-nlp: Neural architecture search benchmark for natural language processing. *IEEE Access*, 10:45736–45747, 2022. doi: 10.1109/ACCESS.2022.3169897.
- [38] A. Mehrotra, A. Ramos, S. Bhattacharya, Ł. Dudziak, R. Vipera, T. Chau, M. Abdelfattah, S. Ishtiaq, and N. Lane. NAS-Bench-ASR: Reproducible Neural Architecture Search for Speech Recognition. In *The Ninth International Conference on Learning Representations (ICLR'21)*, 2021.
- [39] X. Dong, L. Liu, K. Musial, and B. Gabrys. NATS-Bench: Benchmarking NAS algorithms for architecture topology and size. In K. M. Lee, editor, *IEEE Transactions on Pattern Analysis and Machine Intelligence'21*, pages 3634–3646. IEEE Computer Society, 2021.
- [40] C. Li, Z. Yu, Y. Fu, Y. Zhang, Y. Zhao, H. You, Q. Yu, Y. Wang, C. Hao, and Y. Lin. HW-NAS-Bench: Hardware-Aware Neural Architecture Search Benchmark. In *The Ninth International Conference on Learning Representations (ICLR'21)*, 2021.
- [41] Y. Duan, X. Chen, Xu H, Z. Chen, X. Liang, T. Zhang, and Z. Li. TransNAS-Bench-101: Improving Transferability and Generalizability of Cross-Task Neural Architecture Search. In *Proceedings of the International Conference on Computer Vision and Pattern Recognition (CVPR'21)*, pages 5251–5260. Computer Vision Foundation and IEEE Computer Society, IEEE, 2021.
- [42] K. Eggensperger, M. Feurer, F. Hutter, J. Bergstra, J. Snoek, H. Hoos, and K. Leyton-Brown. Towards an empirical foundation for assessing Bayesian optimization of hyperparameters. In M. Hoffman, J. Snoek, N. de Freitas, and M. Osborne, editors, *NeurIPS Workshop on Bayesian Optimization in Theory and Practice (BayesOpt'13)*, 2013.
- [43] K. Eggensperger, F. Hutter, H. Hoos, and K. Leyton-Brown. Efficient benchmarking of hyperparameter optimizers via surrogates. In B. Bonet and S. Koenig, editors, *Proceedings of the Twenty-ninth AAAI Conference on Artificial Intelligence (AAAI'15)*, pages 1114–1120. AAAI Press, 2015.
- [44] J. Kiili, E. Laaksonen, R. Turner, D. Eriksson, S. Park, M. Mccourt, Z. Xu, and I. Guyon. Black box optimization challenge, 2020.

- [45] R. Turner and D. Eriksson. Bayesmark: Benchmark framework to easily compare Bayesian Optimization methods on real machine learning tasks. github.com/uber/bayesmark, 2019.
- [46] A. Klein and F. Hutter. Tabular benchmarks for joint architecture and hyperparameter optimization. *CoRR*, abs/1905.04970, 2019.
- [47] L. Zimmer, M. Lindauer, and F. Hutter. Auto-Pytorch: Multi-fidelity metalearning for efficient and robust AutoDL. *tpami*, 43:3079–3090, 2021.
- [48] L. Choshen, Y. Zhang, and J. Andreas. A hitchhiker’s guide to scaling law estimation, 2025.
- [49] N. Muennighoff, A. M. Rush, B. Barak, T. L. Scao, A. Piktus, N. Tazi, S. Pyysalo, T. Wolf, and C. Raffel. Scaling data-constrained language models, 2025.
- [50] S. McLeish, J. Kirchenbauer, D. Y. Miller, S. Singh, A. Bhatele, M. Goldblum, A. Panda, and T. Goldstein. Gemstones: A model suite for multi-faceted scaling laws, 2025.
- [51] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *The Seventh International Conference on Learning Representations (ICLR’19)*. ICLR, 2019.
- [52] LAION. Releasing re-laion-5b: Transparent iteration on laion-5b with additional safety fixes. <https://laion.ai/blog/re-laion-5b/>, 2024. Accessed: 2024-08-30.
- [53] N. Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [54] B. Zhang and R. Sennrich. Root mean square layer normalization. In *Proceedings of the 32nd International Conference on Advances in Neural Information Processing Systems (NeurIPS’19)*, 2019.
- [55] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.
- [56] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. Hellaswag: Can a machine really finish your sentence? In J. Burstein, C. Doran, and T. Solorio, editors, *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 4791–4800. Association for Computational Linguistics, 2019.
- [57] Y. Bisk, R. Zellers, J. Gao, Y. Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In F. Rossi, V. Conitzer, and F. Sha, editors, *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439. Association for the Advancement of Artificial Intelligence, AAAI Press, 2020.
- [58] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2381–2391. Association for Computational Linguistics, 2018.
- [59] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [60] M. Roemmele, C. A. Bejan, and A. S Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *AAAI spring symposium: logical formalizations of commonsense reasoning*, pages 90–95, 2011.
- [61] S. Y. Gadre, G. Ilharco, A. Fang, J. Hayase, G. Smyrnis, T. Nguyen, R. Marten, M. Wortsman, D. Ghosh, J. Zhang, et al. Datacomp: In search of the next generation of multimodal datasets. In *Advances in Neural Information Processing Systems*, 2023.
- [62] K. Karkkainen and J. Joo. Fairface: Face attribute dataset for balanced race, gender, and age for bias measurement and mitigation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2021.

- [63] Z. Zhang, Y. Song, and H. Qi. Age progression/regression by conditional adversarial autoencoder. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.
- [64] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Proceedings of the 32nd International Conference on Advances in Neural Information Processing Systems (NeurIPS'19)*, 2019.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: See Section 2 for the scaling analysis methodology, Section 3 for surrogate benchmark construction, Section 4 for the evaluation of ScAn methodologies, and Section 4.3 for the experimental results.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: See Section 6.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: See Section 3, 4 and Appendix A, B, C.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The paper does provide open access to the data and code, which are in synchronization with supplemental material.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: See Section 3 and Appendix A and B.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Standard error of the mean is shown in figures in Section 4.3, and in Appendix E.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).

- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: See Table 1 and Table 11.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: Our research conforms to the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: See Appendix D.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification:

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Existing datasets, codebases, and evaluation suites used in this work are documented in the released repositories, including references to their respective licenses and terms of use where applicable.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: Documentation and implementation details for the released benchmark assets are provided in the repository: https://anonymous.4open.science/r/scan_bench_suite-3E3D.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification:

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification:

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: The proposed methodology does not use LLMs as a core methodological component. They are only one of the benchmarked model families.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

A Foundation model pipeline details

A.1 ScAn spaces

The ScAn search space is designed to jointly explore both architectural scale and training hyperparameters. Below we explain the search space for each family.

A.1.1 LLM

Our space comprises nine dimensions organized into two groups: hyperparameters (learning rate, weight decay, β_1 , β_2 , cooldown fraction) and scale parameters (embedding size - d_{model} , number of layers, number of heads, token budget).

The ranges for each parameter are informed by established scaling law literature. For architectural dimensions, we follow the model size progressions studied by Hoffmann et al. [8], covering models from approximately 16M to 1B parameters. The token budget range spans sub-optimal to over-trained regimes relative to compute-optimal scaling. Hyperparameters are drawn from ranges mentioned in the Step Law [11], which provides scaling relationships for batch size and learning rate. The AdamW [51] momentum coefficients (β_1 , β_2) and weight decay bounds reflect common practice in transformer training. Full scaling and hyperparameter ranges are shown in Table 5. Notably, batch size is not sampled directly, instead we derive it from the token budget using the Step Law’s prescribed relationship, ensuring compute-optimal throughput for each configuration, with ranges depicted in Table 6.

Table 5: LLM ScAn Space Configuration

Parameters	Type	Range / Set	Sampling
Learning rate	Float	$[1 \times 10^{-5}, 1 \times 10^{-2}]$	Log
Weight decay	Float	$[1 \times 10^{-3}, 0.2]$	Log
β_1	Float	$[0.8, 0.99]$	Log
β_2	Float	$[0.9, 0.999]$	Log
Cooldown fraction	Float	$[0.0, 0.3]$	Linear
Embedding size (d_{model})	Categorical	$\{256, 320, \dots, 1600\}$	Linear
Number of layers	Categorical	$\{4, 5, \dots, 30\}$	Linear
Number of heads	Categorical	$\{6, 8, \dots, 40\}$	Linear
Number of tokens	Float	$[2 \times 10^8, 4 \times 10^{10}]$	Log

Table 6: Batch size heuristic ranges derived from Step Law.

Token Budget	Global Batch Size
$[0, 10^8)$	32768
$[10^8, 10^9)$	65536
$[10^9, 10^{10})$	131072
$[10^{10}, 2 \times 10^{10})$	262144
$[2 \times 10^{10}, 4 \times 10^{10})$	524288

A.1.2 VLM

Our search space comprises three scale parameters: vision width, text width, and the number of training samples, along with six optimization hyperparameters: learning rate, weight decay, optimizer parameters (β_1 , β_2 , ϵ), and the warmup fraction. Table 7 summarizes the full space and corresponding ranges (note that training samples is in millions), while the sampling strategy is described in the next section. The parameter bounds are guided by prior work [5, 9, 10].

In addition to the search dimensions, several design choices are fixed across all configurations, and results are therefore conditioned on these settings. Table 8 summarizes the fixed choices. All remaining parameters not listed in the table follow defaults from [9], such as vocabulary size and text context length. The attention head dimension (marked with * in the table) is set to 64, except for the

Table 7: VLM ScAn Space Configuration

Parameters	Type	Range / Set	Sampling
Learning rate	Float	$[1 \times 10^{-5}, 5 \times 10^{-2}]$	Log
Weight decay	Float	$[1 \times 10^{-6}, 2 \times 10^{-1}]$	Log
Warmup fraction	Float	$[0.0, 0.75]$	Linear
β_1	Float	$[0.9, 0.99]$	Log
β_2	Float	$[0.95, 0.999]$	Log
ϵ	Float	$[1 \times 10^{-8}, 1 \times 10^{-6}]$	Log
Training samples (M)	Float	$[0.6, 120.0]$	Warped log ($\alpha = 0.5$)
Vision width	Categorical	$\{32, 64, \dots, 512\}$	Weighted categorical ($\alpha = 0.5$)
Text width	Categorical	$\{32, 64, \dots, 512\}$	Weighted categorical ($\alpha = 0.5$)

smallest models with embedding size 32, where it is reduced to 32. The number of attention heads is computed as the embedding size of the respective tower divided by 64, with a minimum of one head for the smallest configurations. This applies to both the vision and text towers.

Table 8: Fixed design choices across all VLM configurations.

Component	Choice
Optimizer	AdamW
Vision backbone	Vision Transformer (ViT)
Image patch size	32
Precision	Mixed precision
Layers (vision/text)	12 / 12
Shared embedding dimension(joint space)	384
Attention head dimension	64*
Learning rate schedule	Cosine

The batch size is not fixed. Instead, we derive a heuristic inspired by [10] on their results on ReLAION-1.4B [52] dataset. We analyze Pareto-optimal configurations under a cosine learning rate schedule and observe that global batch size increases consistently with training compute. Based on this observation, we construct a mapping from the number of training samples to the corresponding global batch size. Table 9 summarizes the resulting heuristic. An alternative mapping from FLOPs to batch size was considered but leads to undesirable regimes in our setting with smaller models. In particular, very small models trained on a large number of samples would receive extremely small batch sizes, resulting in inefficient and excessively long training times.

Table 9: Batch size heuristic derived from Pareto-optimal OpenCLIP configurations.

Training Samples	Global Batch Size
$[0, 3.4\text{M})$	512
$[3.4\text{M}, 14.6\text{M})$	2,048
$[14.6\text{M}, 70.4\text{M})$	4,096
$[70.4\text{M}, 145.92\text{M})$	8,192
$[145.92\text{M}, 340.48\text{M})$	16,384

A.2 Sampling strategies

Across all model families, most parameters are sampled via random search, with continuous parameters drawn either uniformly or log-uniformly depending on their scale. In addition, some parameters follow custom sampling strategies, which are described in detail in the following sections. The ScAn space tables introduced earlier already include a sampling column that provides a brief overview of these strategies.

A.2.1 LLM

Configurations are sampled from the search space defined in Table 5 following the general procedure described above. Each sampled configuration fully specifies the model architecture (embedding size, depth, number of heads), the training budget (token count), and the hyperparameters (learning rate, weight decay, β_1 , β_2 , cooldown fraction).

A.2.2 VLM

Hyperparameters are sampled according to the general procedure described above (Table 7), with additional specific strategies detailed below.

The number of training samples N is drawn from a warped log-uniform distribution:

$$N = N_{\min} \left(\frac{N_{\max}}{N_{\min}} \right)^{u^\alpha}, \quad u \sim \mathcal{U}(0, 1), \quad (2)$$

where $N_{\min} = 6 \times 10^5$, $N_{\max} = 1.2 \times 10^8$, and $\alpha = 0.5$.

A standard log-uniform distribution would allocate equal probability mass to each order of magnitude, resulting in half of the configurations being sampled below 6×10^6 samples. This was found to be overly restrictive. The warped formulation ($\alpha < 1$) provides a controlled trade-off by relaxing the equal-per-decade constraint of log-uniform sampling.

Architectural parameters, namely vision and text width, are selected from the discrete set $\{32, 64, 128, 192, 256, 320, 384, 448, 512\}$. To bias sampling toward smaller model sizes, we assign a weight to each candidate based on its index. Let $i \in \{1, \dots, K\}$ denote the index of a candidate (ordered from smallest to largest). The weights are defined as

$$w_i = \frac{1}{i^\alpha}, \quad (3)$$

with $\alpha = 0.5$, such that smaller widths receive higher weights. These weights are normalized to obtain a probability distribution:

$$P(i) = \frac{w_i}{\sum_{j=1}^K w_j}, \quad (4)$$

from which the final width is sampled.

A.3 Multi-fidelity modeling and checkpointing

We record intermediate checkpoints for each configuration to enable multi-fidelity modeling in the surrogate benchmarks. Each checkpoint corresponds to a different stage of training. To capture this, we define a fidelity variable `training_progress` $\in [0, 1]$, representing normalized training progress, where 0 corresponds to initialization and 1 to the fully trained model. Each checkpoint is mapped to a value of `training_progress` based on its relative position within the training process. This yields multiple observations per configuration at different fidelity levels, enabling the surrogate to model performance as a function of both configuration and training progress.

A.3.1 LLM

We determine the number of training steps for each model based on a fixed token budget. Specifically,

$$N_{\text{steps}} = \left\lfloor \frac{T}{S \cdot B_{\text{global}}} \right\rfloor, \quad (5)$$

where T denotes the target token budget, $S = 2,048$ is the sequence length, and

$$B_{\text{global}} = \text{micro_batch_size} \times \text{gradient_accumulation_steps} \times \text{num_gpus} \quad (6)$$

is the global batch size.

Let $N_{\text{steps}}^{(m)}$ denote the number of steps for model m , and let :

$$N_{\text{steps}}^{\max} = \max_m N_{\text{steps}}^{(m)} \quad (7)$$

be the maximum number of steps across all models.

The number of checkpoints assigned to each model is then given by:

$$N_{\text{ckpt}}^{(m)} = \max \left(2, \left\lceil 10 \cdot \frac{N_{\text{steps}}^{(m)}}{N_{\text{steps}}^{\max}} \right\rceil \right). \quad (8)$$

Thus, the model with the largest number of training steps is assigned 10 checkpoints, while all other models receive between 2 and 10 checkpoints, proportional to their number of training steps.

A.3.2 VLM

The number of epochs, which determines the number of recorded checkpoints, is computed as:

$$E = \max \left(2, \left\lceil \frac{N_{\text{planned}}}{N_{\text{max per epoch}}} \right\rceil \right), \quad (9)$$

where N_{planned} denotes the planned training budget, defined as the number of training samples. The constant $N_{\text{max per epoch}}$ controls the frequency of checkpointing, and is set to 6 million. For each configuration we have 2 to 10 checkpoints.

A.4 Model training and evaluation

A.4.1 LLM

Training Every configuration is trained from scratch on a designated subset of the SlimPajama [28] dataset using a decoder-only transformer with AdamW [51] as an optimizer, SwiGLU [53] activations, RMSNorm [54], weight tying, a vocabulary size of 50 277 and a sequence length of 2 048. Training follows a warmup-stable-decay (WSD) [55] learning rate schedule. Full training information is depicted in Table 10. The total number of training steps is determined by the token budget and global batch size like mentioned above in equation (5).

Table 10: Fixed training choices across all LLM configurations.

Component	Choice
Optimizer	AdamW
Vocabulary size	50277
Sequence length	2048
Activation function	SwiGLU
Learning rate schedule	WSD
Normalization	RMSNorm
Weight tying	True

Evaluation At each evaluation step, upstream performance is measured via validation and test loss on the respective SlimPajama [28] split, while downstream performance is evaluated on six benchmarks (HellaSwag [56], PIQA [57], OpenBookQA [58], ARC Easy [59], ARC Challenge [59], COPA [60]) using the respective few-shot settings.

A.4.2 VLM

Training Models are trained using the AdamW [51] optimizer with a global batch size determined by the heuristic described earlier and gradient accumulation set to 1. Training is performed in a distributed setting using local loss computation instead of constructing the full global similarity matrix.

Evaluation Each trained model is evaluated on separate validation and test sets to obtain performance metrics. Both splits are drawn from pre-training dataset, with 100,000 samples each. Evaluation is performed with a fixed batch size of 128 to ensure comparability across configurations.

Table 11: Compute cost (GPU hours) for data collection across different GPU types. The total column reports the sum of training and evaluation costs across both VLM and LLM benchmarks.

GPU	Training		Downstream		Upstream		Total
	VLM	LLM	VLM	LLM	VLM	LLM	
RTX 2080 Ti	11.84	-	731.84	-	11.39	-	755.07
RTX 3080	142.42	-	373.53	-	13.47	-	529.42
L40	0.44	-	-	-	9.84	-	10.28
A100 40GB	3986.02	11042.45	828.75	935.35	198.66	3972.34	20962.55
H100 94GB	6653.11	26870.62	859.65	-	159.61	348.27	34891.26

For downstream evaluation, we use the DataComp evaluation suite [61], which comprises 40 datasets evaluated in a zero-shot setting without additional training. The suite spans multiple task categories, including classification, retrieval, robustness, and fairness benchmarks. As individual tasks report different evaluation metrics, we model the primary metric specified for each task by [61]. For FAIRFACE [62] and UTKFACE [63], which do not define a single primary metric, we use `acc_race_avg`.

B Surrogate modeling details

B.1 Surrogate candidates

We adopt the surrogate modeling setup from HW-GPT-Bench [23], including AutoGluon, XGBoost, LightGBM, and mixed ensembles, as described in Appendix C.2 of the original work.

We introduce three modifications. First, we exclude the XGBoost learning rate 1.0, as it did not improve predictive performance in our setting. Second, we use AutoGluon v1.5 with default configurations instead of manually specified hyperparameters, as we observed no meaningful differences in accuracy. For AutoGluon, we set the training time budget to 30 minutes for LLMs and 4 hours for VLMs. Third, we exclude FASTAI models in AutoGluon due to training stability issues.

In addition, we include TabPFN [34] v2.5 as an additional surrogate candidate. TabPFN has demonstrated strong performance on low-data tabular tasks, which aligns with our setting. Moreover, its in-context learning capability removes the need for explicit training, allowing efficient reuse across multiple downstream targets without training separate models for each metric.

B.2 Data splitting

The dataset contains multiple checkpoints per configuration. To avoid information leakage, splitting is performed at the configuration level, assigning each configuration and all its checkpoints entirely to either the training or test set.

For VLMs, we perform an 80/20 train test split over configurations. To ensure coverage across different compute scales, configurations are partitioned into four equally sized groups based on their training compute, and a stratified split is performed within each group. This ensures that both training and test sets include configurations from all compute ranges. On the LLMs case, we use a 90/10 random split without additional balancing.

Additionally, in VLMs we build an additional failure prediction dataset, where each configuration is assigned a binary label indicating whether it diverged during training. Configurations are randomly split into training and test sets using a 80/20 split. This dataset is defined at the configuration level, and checkpoints are not considered.

B.3 Evaluation metrics

We adopt the accuracy metrics from HW-GPT-Bench [23], including Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), Median Absolute Error (MDAE), Mean Absolute Relative Percentage Deviation (MARPD), coefficient of determination (R^2), Pearson correlation (R), and Spearman rank correlation, and omit calibration metrics to focus solely on predictive performance.

Table 12: Surrogate performance across model families on held-out final configurations.

Family	Surrogate	RMSE ↓	MAE ↓	MDAE ↓	MARPD ↓	R^2 ↑	R ↑	Corr. ↑
VLM	TabPFN	0.21	0.06	0.02	2.69	0.96	0.98	0.98
	AutoGluon	0.26	0.12	0.07	5.79	0.94	0.97	0.98
	XGB	0.35	0.20	0.13	9.11	0.90	0.95	0.96
	Mix	0.37	0.21	0.14	10.11	0.89	0.95	0.95
	LGB	0.42	0.29	0.24	14.88	0.86	0.95	0.96
LLM	TabPFN	0.27	0.08	0.01	3.02	0.77	0.88	0.95
	AutoGluon	0.30	0.11	0.02	4.66	0.72	0.85	0.91
	XGB	0.35	0.15	0.04	6.80	0.63	0.80	0.86
	Mix	0.33	0.17	0.07	7.65	0.66	0.82	0.86
	LGB	0.33	0.17	0.08	7.97	0.66	0.83	0.87

Table 12 summarizes surrogate performance across model families and metrics, using the respective train test split and reporting results on upstream performance for each model family.

B.4 Divergence prediction (VLMs)

As described in B.2 data splitting procedure, we construct a dataset for modeling failed runs. Based on this data, we train a binary classifier that predicts whether a configuration will diverge. Empirically, divergence is primarily caused by extreme hyperparameter settings, in particular high learning rates. We therefore use a simple model based on gradient boosted decision trees.

Specifically, we use an ensemble of XGBoost classifiers. Each model in the ensemble is an XGBoost classifier with tree-based architecture, trained with varying maximum depth $\{3, 5, 9\}$, number of estimators $\{300, 500, 800\}$, and learning rates $\{0.01, 0.05, 0.1\}$.

At query time, configurations predicted to diverge are filtered out and no performance prediction is returned.

C Surrogate API

The surrogate API provides a unified interface for querying configurations at arbitrary training progress. Given a configuration and a training progress value, the API returns upstream and downstream metrics predicted by surrogate models, along with predictive uncertainty estimates and additional quantities computed from the configuration, including cumulative training compute (FLOPs) and model parameter counts. Table 13 provides an overview of the interface, conditioned on the configuration not being predicted to diverge in the VLM case. Configurations predicted to fail do not return performance outputs, but only auxiliary quantities such as FLOPs and parameter counts.

C.1 Predictive uncertainty

The API encodes predictive uncertainty via TabPFN’s quantile outputs, defined as the difference between the 90th and 10th percentiles of the predictive distribution.

C.2 Parameter count

LLMs The total number of trainable parameters N is computed as:

$$N = L(4d^2 + 3dh + 2d) + 2dV + d, \quad (10)$$

where L is the number of layers, d the model embedding dimension, V the vocabulary size, and h the hidden dimension of the feed-forward network, defined as:

$$h = 256 \left\lceil \frac{8d/3}{256} \right\rceil. \quad (11)$$

VLMs Model parameter counts are computed as the total number of trainable parameters. We further report separate parameter counts for the vision encoder and the text encoder. For the text

Table 13: Surrogate API query interface. Given a configuration and a fidelity value, the API returns performance predictions and additional metadata.

Category	Field	Description
Input	Configuration	Sampled from ScAn space A.1
	Training progress	Fidelity in $[0, 1]$ indicating training progress
Output	Upstream metrics	Validation or test loss
	Downstream metrics	Defined in A.4
	FLOPs	Cumulative training compute up to queried fidelity
	Model parameters	Number of model parameters
	Uncertainty	Surrogate model uncertainty

encoder, we additionally distinguish non-embedding parameters, which comprise the transformer layers, final layer normalization, and text projection layers, excluding token embeddings.

C.3 FLOPs estimation

LLMs We approximate the total training compute C in FLOPs by accounting for both forward and backward passes. Each parameter contributes approximately 2 FLOPs per token in the forward pass, and the backward pass is estimated as twice the forward pass, yielding an overall factor of 6. The total compute is thus given by:

$$C = 6T \left[L(4d^2 + 2Sd + 3dh) + dV \right], \quad (12)$$

where T denotes the total number of training tokens and S the sequence length.

VLMs We follow the OpenCLIP implementation for FLOPs estimation. Specifically, we use the PyTorch [64] `torch.utils.flop_counter` profiler to measure the per-sample FLOPs of a forward pass. Total training compute is then obtained by multiplying the per-sample FLOPs by the total number of training samples.

D Broader impacts

Our goal is to make scaling law research for VLMs and LLMs more efficient, reproducible, and comparable. Surrogate benchmarks enable systematic evaluation of scaling methodologies without repeated large-scale training, reducing computational cost and carbon emissions. In practice, methods are often compared under different datasets, training procedures, and compute budgets, making it unclear which method performs better. Our approach enables fair and controlled comparison across techniques, while lowering the barrier to entry for large-scale model research. To the extent that this improves scaling laws, it may contribute to the development of more capable foundation models. Such progress can have both positive and negative societal impacts, as more powerful models can enable beneficial applications but may also be misused, for example to generate misleading content, amplify biases, or support large-scale surveillance. These risks arise from the underlying models rather than the benchmark itself.

E ScAn Methodology

E.1 Chinchilla-based data acquisition design choices

We adapt generalized, heuristic-free baselines. For example, in Chinchilla acquisition approaches, rather than relying on tuned priors, we evaluate the allocation strategy across 10 random seeds, where each seed operates under a fixed λ_o uniformly sampled from Λ_o . Furthermore, because the original work lacks a formal prescription for generating the discrete grid of evaluated models, we systematize this generation process. We construct a monotonically increasing sequence of configurations, $\lambda_{c,t_i} \leq \lambda_{c,t_{i+1}}$, utilizing Sobol sampling to iteratively select which specific dimension j of the scaling parameters λ_c is augmented at each step. The corresponding pseudo-code in more detail exists in Algorithm 1 and Algorithm 2.

E.2 CARBS data acquisition design choices

We apply this same strict, prior-free constraint to the CARBS baseline. While we maintain the core acquisition behavior, the standard CARBS implementation relies on a manually specified, prior-informed search center. To eliminate this human bias, we replace it with a systematic, automated initialization protocol, ensuring a completely equitable comparison against the Chinchilla baselines. We set most of the hyperparameters to the defaults used in their original source code, providing only the search center. If a hyperparameter is in LogSpace, we use the $\sqrt{\min \cdot \max}$ value of its domain for the search center; otherwise, we get the mean of the min and max and use it as the center, along with defining the initial search radius.

Algorithm 1 Systematized Iso-Parameter Acquisition (Single Experiment)

Require: Random seed k , non-scaling parameter space Λ_o , compute budget $[C_{\min}, C_{\max}]$
Require: Token scaling multipliers M (e.g., $\{5, 30, 60, 80\}$ for LLM)

- 1: Initialize experiment environment with random seed k
- 2: Initialize acquired dataset $\mathcal{D} \leftarrow \emptyset$
- 3: Sample fixed hyperparameters uniformly: $\lambda_o \sim \text{Uniform}(\Lambda_o)$
- 4: Initialize configuration sequence $\mathcal{S}_c \leftarrow \emptyset$ and base model size λ_{c,t_0}
 - ▷ Phase 1: Generate monotonically increasing model size grid
- 5: **while** grid size < target horizon (e.g., 16) **do**
- 6: Use Sobol sampling to select dimension j of λ_c to augment
- 7: $\lambda_{c,t_{i+1}} \leftarrow \text{Augment}(\lambda_{c,t_i}, \text{dim} = j)$
- 8: $\mathcal{S}_c \leftarrow \mathcal{S}_c \cup \{\lambda_{c,t_{i+1}}\}$
- 9: **end while**
 - ▷ Phase 2: Evaluate combinations across fixed token multipliers
- 10: **for each** model configuration $\lambda_c \in \mathcal{S}_c$ **do**
- 11: **for each** multiplier $m \in M$ **do**
- 12: Compute token count $D \leftarrow m \times \text{Size}(\lambda_c)$
- 13: Estimate total compute cost C for (λ_c, D)
 - ▷ Phase 3: Filter by compute bounds and execute
- 14: **if** $C_{\min} \leq C \leq C_{\max}$ **then**
- 15: $L \leftarrow \text{TrainAndEvaluate}(\lambda_c, \lambda_o, D)$
- 16: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(C, \lambda_c, \lambda_o, D, L)\}$
- 17: **end if**
- 18: **end for**
- 19: **end for**
- 20: **return** $\mathcal{D}$

E.3 Loss Extrapolation

Looking at the performance of each data acquisition strategy with the Kaplan and Chinchilla 3 approaches in Figures 7 and 8, we show that the Chinchilla 3 approach models the optimal trajectory better in the OpenCLIP benchmark across all acquisition methods. However, the regret ranges for both approaches in the LLM benchmark are comparable, demonstrating the overall superior performance of Chinchilla 3 in modeling optimal loss.

E.4 Coarse Extrapolation

Here, we demonstrate the model size prediction regret of different data acquisition strategies using the Chinchilla 3 extrapolation technique, highlighting its superior performance on the LLM benchmark due to the compute cost estimation embedded within its parametric loss function.

E.5 Robustness of Extrapolation at $10\times$ Target Scale

We have already observed the prediction performance given the $20 \times C_{\max}$. In Figures 10, 11, 12, and 13 we illustrate a consistent prediction for $10\times$ setting compared to $20\times$ setting which is theoretically expected. Because the underlying prediction techniques model scaling behavior using power laws of the form C_{target}^α , increasing the target compute budget by a scalar multiple effectively

Algorithm 2 Systematized Iso-FLOP (Chinchilla 2) Acquisition

Require: Random seed k , non-scaling parameter space Λ_o , compute bounds $C_{\min}, C_{\max}$
Require: Number of target FLOP steps N_{steps} , compute cost tolerance ϵ

- 1: Initialize experiment environment with random seed k
- 2: Initialize acquired dataset $\mathcal{D} \leftarrow \emptyset$
- 3: Sample fixed hyperparameters uniformly: $\lambda_o \sim \text{Uniform}(\Lambda_o)$
- 4: Generate model configuration pool $\mathcal{S}_c$ via Sobol sampling
 - ▷ Phase 1: Generate logarithmic compute budget grid
- 5: $C_{\text{grid}} \leftarrow \text{LogSpace}(\log_{10}(2 \cdot C_{\min}), \log_{10}(C_{\max}), N_{\text{steps}})$
 - ▷ Phase 2: Iterate over fixed target budgets and model sizes
- 6: **for each** target compute budget $C_{\text{target}} \in C_{\text{grid}}$ **do**
- 7: **for each** model configuration $\lambda_c \in \mathcal{S}_c$ **do**
 - ▷ Phase 3: Binary search for required token count D
- 8: $D_{\text{low}} \leftarrow 1, D_{\text{high}} \leftarrow D_{\text{limit}}$
- 9: $D_{\text{match}} \leftarrow \emptyset$
- 10: **while** $D_{\text{low}} \leq D_{\text{high}}$ **do**
- 11: $D_{\text{mid}} \leftarrow \lfloor (D_{\text{low}} + D_{\text{high}})/2 \rfloor$
- 12: $C_{\text{curr}} \leftarrow \text{EstimateCost}(\lambda_c, D_{\text{mid}})$
- 13: **if** $|C_{\text{curr}} - C_{\text{target}}| < \epsilon$ **then**
- 14: $D_{\text{match}} \leftarrow D_{\text{mid}}$
- 15: **break** ▷ Match found within tolerance
- 16: **else if** $C_{\text{curr}} < C_{\text{target}}$ **then**
- 17: $D_{\text{low}} \leftarrow D_{\text{mid}} + 1$
- 18: **else**
- 19: $D_{\text{high}} \leftarrow D_{\text{mid}} - 1$
- 20: **end if**
- 21: **end while** ▷ Phase 4: Execute if a valid configuration is resolved
- 22: **if** $D_{\text{match}} \neq \emptyset$ **then**
- 23: $L \leftarrow \text{TrainAndEvaluate}(\lambda_c, \lambda_o, D_{\text{match}})$
- 24: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(C_{\text{target}}, \lambda_c, \lambda_o, D_{\text{match}}, L)\}$
- 25: **end if**
- 26: **end for**
- 27: **end for**
- 28: **return** $\mathcal{D}$

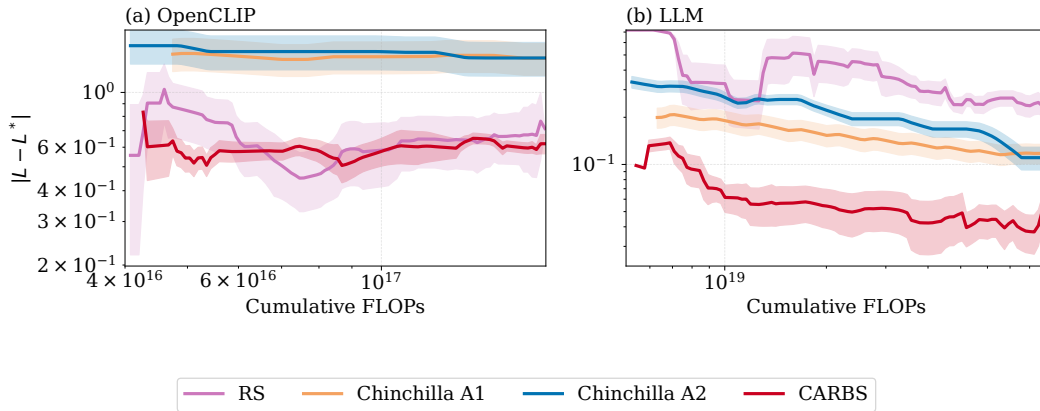


Figure 7: **Loss extrapolation regret over trajectory using Kaplan strategy:** Illustrating the loss prediction regret for all the data acquisition strategies using Kaplan extrapolation approach.

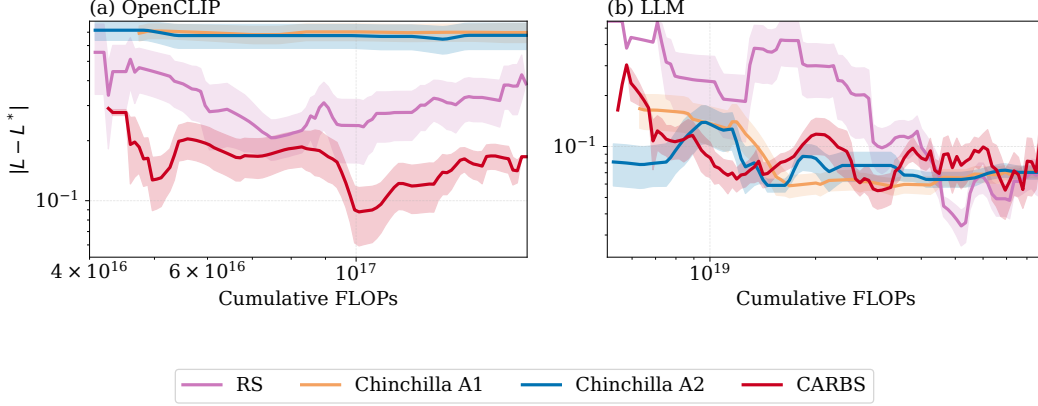


Figure 8: **Loss extrapolation regret over trajectory using Chinchilla 3 strategy:** Illustrating the loss prediction regret for all the data acquisition strategies using Chinchilla 3 extrapolation approach.

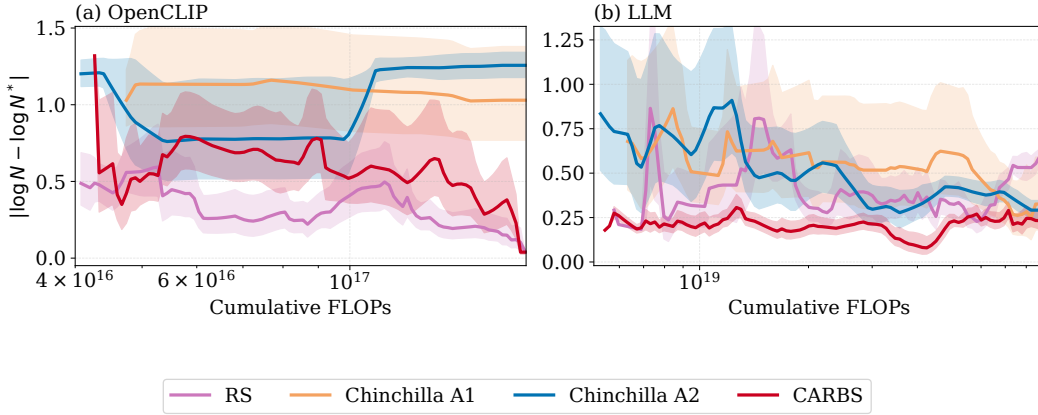


Figure 9: **Model size extrapolation regret over trajectory using the Chinchilla 3 strategy.** The plots display the absolute logarithmic regret, defined as $|\log N - \log N^*|$, which measures the deviation between the predicted optimal parameter count (N) and the true empirical optimum (N^*) as a function of cumulative profiling FLOPs. Results are shown for (a) the OpenCLIP and (b) LLM benchmarks. Shaded regions denote the standard error across independent seeds. Notably, the CARBS acquisition strategy achieves the lowest final parameter prediction regret across both modalities.

induces a uniform shift along the prediction trajectory in log-log space. Consequently, the relative ranking of the extrapolation techniques remains preserved, provided this shift is synchronous with the shift of the empirical optimal loss to the new target scale. To provide greater transparency for large-scale predictions, we have added new plots detailing the loss and optimal parameter (N^*) prediction trajectories at $10\times$ extrapolation to this section.

E.6 Robustness and Generalizability of the Surrogate Model

Building upon the acquisition-prediction dynamics established in Section 4.3, we conduct two targeted ablation studies to rigorously evaluate the generalizability of our framework. Our objective is to verify that the core findings, specifically the relative method rankings and the interactions between acquisition and extrapolation strategies, are intrinsic to the scaling methods themselves, rather than artifacts of the chosen surrogate model or the training data distribution.

First, we evaluate sensitivity to the surrogate architecture. Figures 16 and 17 illustrate the loss and coarse parameter extrapolation trajectories using an XGBoost surrogate in place of the primary TabPFN model. Evaluated at a highly scaled $20\times$ extrapolation budget, the XGBoost surrogate yields

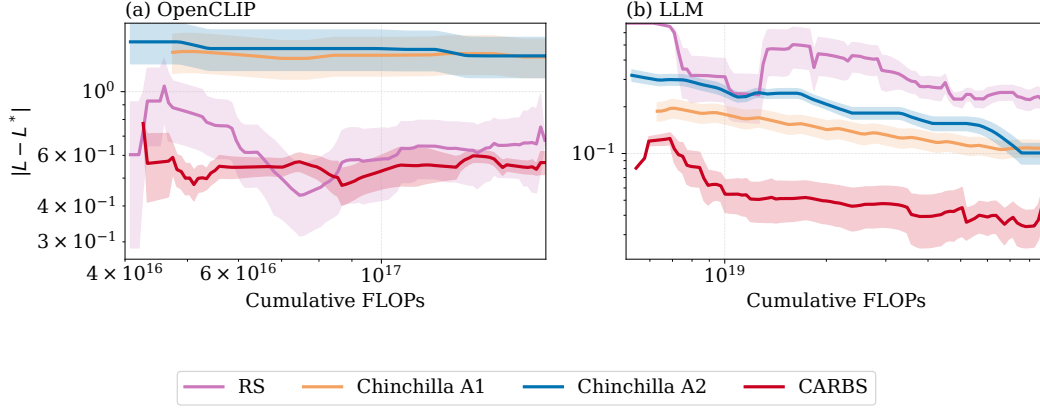


Figure 10: **Loss extrapolation trajectories at $10\times$ target scale using the Kaplan prediction strategy.** This plot illustrates the predicted loss trajectories over cumulative FLOPs across varied acquisition strategies evaluated at $(10 \times C_{\max})$. Consistent with baseline findings, extending the target budget induces a uniform shift in the prediction trajectory in log-log space, preserving the stable, monotonic improvement characteristic of the Kaplan method.

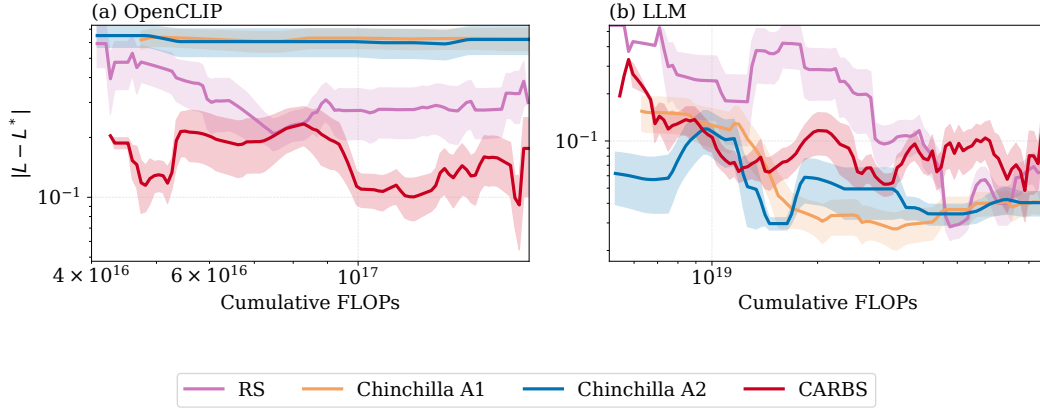


Figure 11: **Loss extrapolation trajectories at $10\times$ target scale using the Chinchilla Approach 3 strategy.** Predicted loss is evaluated at target compute budget of $10 \times C_{\max}$. While the large extrapolation gap naturally shifts the prediction trajectory, the comparative behavior and relative ranking of the Chinchilla Approach 3, including its stronger localized predictions on specific benchmarks, remain preserved and synchronous with the empirical loss trajectory.

remarkably comparable performance trends and interaction effects, confirming that our conclusions are agnostic to the underlying surrogate mechanism.

Second, we assess robustness to data variance. Figures 18 and 19 present the extrapolation results when the TabPFN surrogate is trained independently on two disjoint subsets of the data corpus. Despite the shift in the empirical training distribution, the relative rankings and scaling behaviors remain strictly preserved. Together, these ablations demonstrate that the benchmarking framework’s conclusions generalize reliably across both architectural substitutions and data variations.

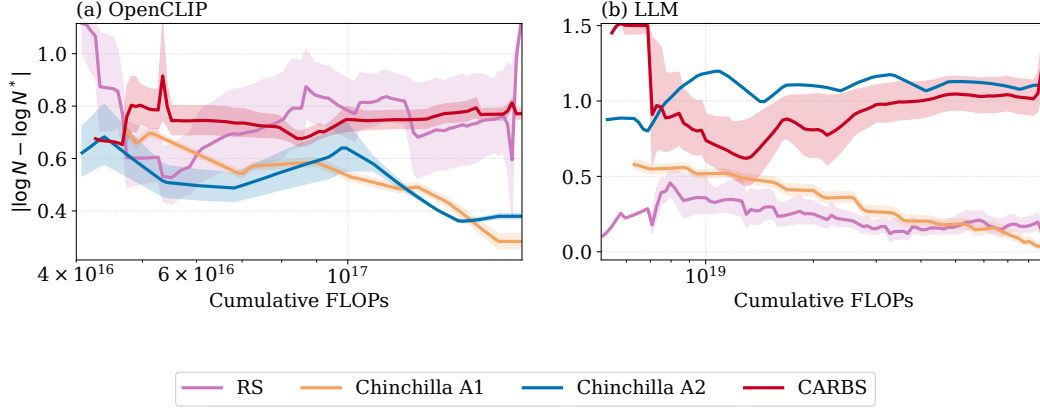


Figure 12: **Robustness of Power-Law model size (N^*) predictions at $10\times$ target scale.** Trajectories of the predicted optimal model size are evaluated against a $10\times$ extrapolation gap across acquisition strategies. The purely empirical Power-Law fit continues to demonstrate strong robustness across diverse architectural domains. The relative ranking of acquisition methods remains entirely consistent with baseline scale results, including the degradation of CARBS due to its localized exploitation bias.

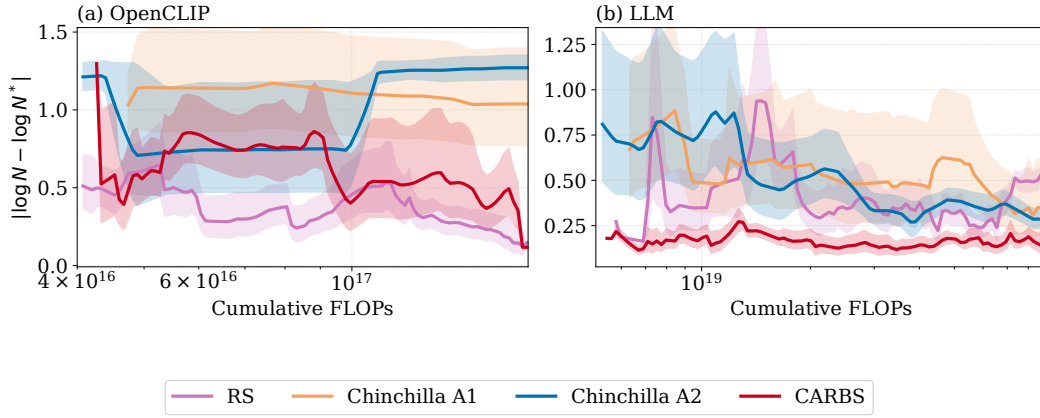


Figure 13: **Model size (N^*) predictions at $10\times$ target scale using the Chinchilla Approach 3 strategy.** This figure highlights the extrapolation of optimal model size. The parametric limitations of Chinchilla Approach 3 persist at this scale; specifically, its rigid reliance on standard compute estimation heuristics continues to cause structural prediction failures on multi-modal architectures like OpenCLIP.

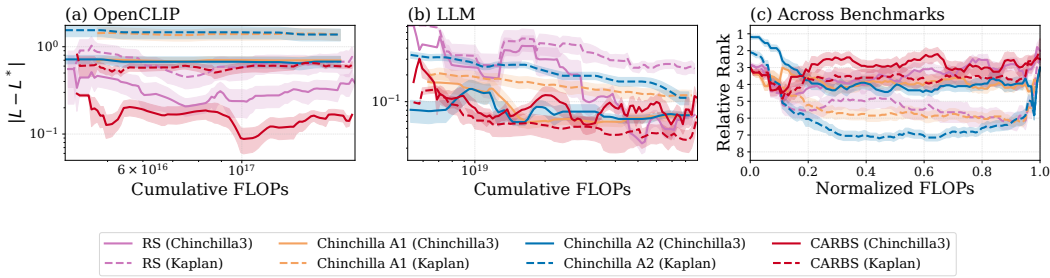


Figure 14: **Interaction between data acquisition strategies and loss prediction methods.** This figure illustrates the interplay between acquisition strategies and prediction techniques (Chinchilla Approach 3 and Kaplan) across the (a) OpenCLIP and (b) LLM benchmarks, alongside (c) their relative performance rankings.

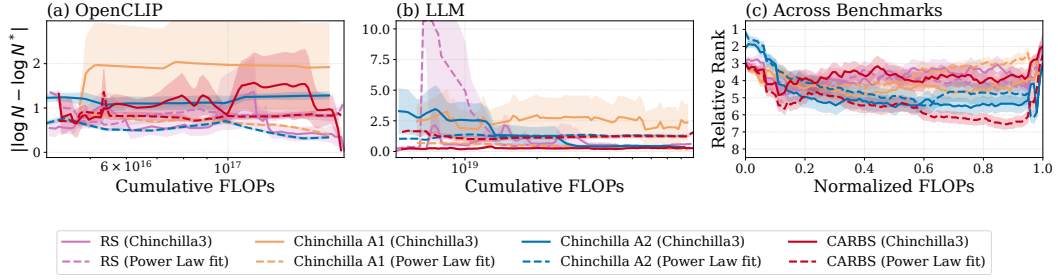


Figure 15: **Interaction between data acquisition strategies and model size prediction methods.** This figure illustrates the interplay between acquisition strategies and prediction techniques (Chinchilla Approach 3 and Power Law fit) across the (a) OpenCLIP and (b) LLM benchmarks, alongside (c) their relative performance rankings.

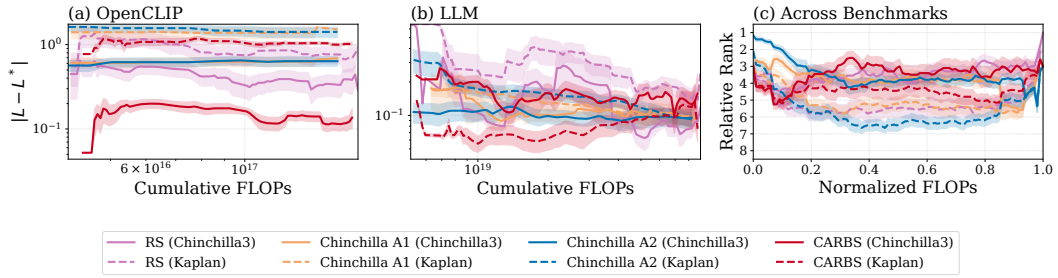


Figure 16: **Interaction between data acquisition strategies and loss prediction methods using an XGBoost surrogate.** This figure illustrates the same interplay between acquisition strategies and prediction techniques (Chinchilla Approach 3 and Kaplan) observed with the primary surrogate across the (a) OpenCLIP and (b) LLM benchmarks, alongside (c) their relative performance rankings.

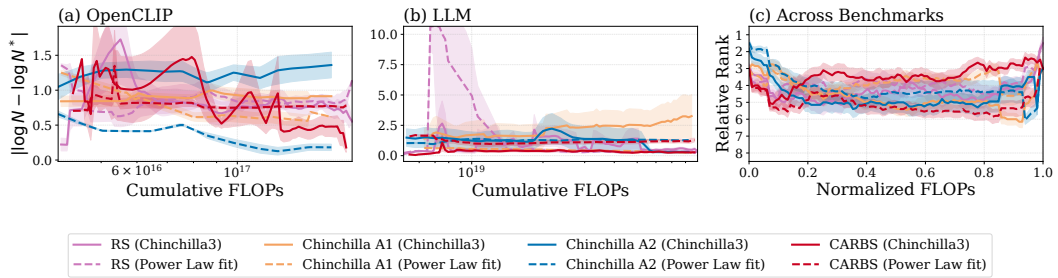


Figure 17: **Interaction between data acquisition strategies and model size prediction methods using an XGBoost surrogate.** This figure illustrates the same interplay between acquisition strategies and prediction techniques (Chinchilla Approach 3 and Power-Law fit) observed with the primary surrogate across the (a) OpenCLIP and (b) LLM benchmarks, alongside (c) their relative performance rankings.

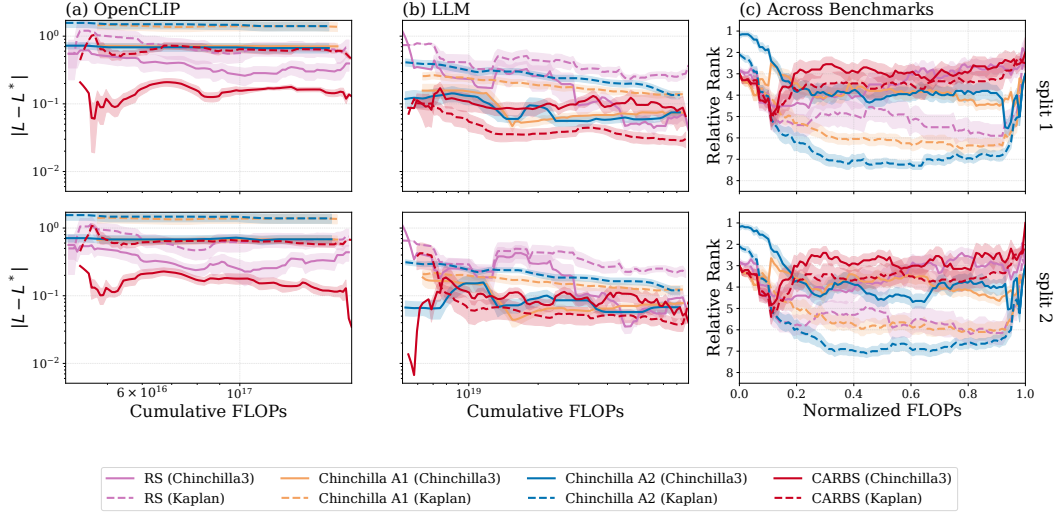


Figure 18: **Interaction between data acquisition strategies and loss prediction methods using the TabPFN surrogate trained on disjoint data subsets.** This figure illustrates that the interplay between acquisition strategies and prediction techniques (Chinchilla Approach 3 and Kaplan) remains consistent when trained on two disjoint subsets of the corpus, evaluated across the (a) OpenCLIP and (b) LLM benchmarks, alongside (c) their relative performance rankings.

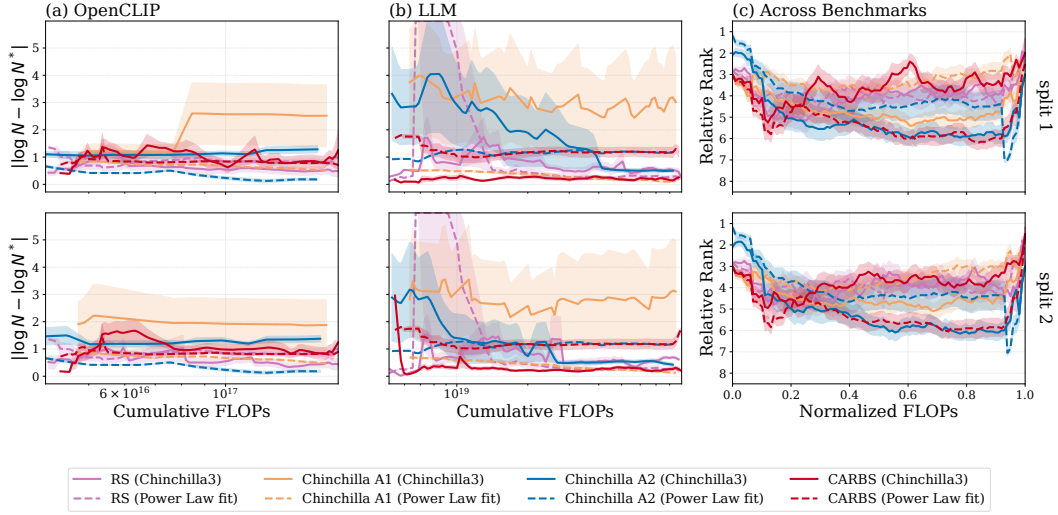


Figure 19: **Interaction between data acquisition strategies and model size prediction methods using the TabPFN surrogate trained on disjoint data subsets.** This figure illustrates that the interplay between acquisition strategies and prediction techniques (Chinchilla Approach 3 and Power-Law fit) remains consistent when trained on two disjoint subsets of the corpus, evaluated across the (a) OpenCLIP and (b) LLM benchmarks, alongside (c) their relative performance rankings.